

Adversarial Attack and Defence Strategies for Deep-Learning-Based IoT Device Classification Techniques

Abhijit Singh^{id} and Biplab Sikdar^{id}, *Senior Member, IEEE*

Abstract—Concurrent advancements in machine learning (ML) and Internet of Things have allowed several interesting interdisciplinary applications, such as classification tasks based on data generated by smart devices for applications, such as security, resource allocation, activity and task classification. However, these applications can be vulnerable to attacks by adversarial examples. The first contribution of this article is the development of a white-box adversarial attack mechanism to generate adversarial examples for data obtained from smart meters installed in residential houses. For the second contribution, we present an analysis to demonstrate that the statistical properties of adversarial datapoints are indistinguishable from those of the true datapoints. The attack is developed specifically for deep-learning-based models used to perform appliance classification in smart home environments. The statistical indistinguishability of the adversarial datapoints from the true datapoints indicates that non ML-based solutions may not be able to tackle the challenge posed by adversarial examples. As the final contribution, we evaluate the effectiveness of defence mechanisms for white-box adversarial attacks on the proposed attack mechanism, and show that while they can reduce the potency of the attack, the original models still remain significantly affected by the adversarial attack. The effectiveness of the proposed techniques is demonstrated on two publicly available data sets: 1) United Kingdom-domestic appliance-level electricity smart meter data set and 2) the Personalized Retrofit Decision Support Tools For U.K. Homes Using Smart Home Technology data set.

Index Terms—Adversarial attacks, cybersecurity, Internet of Things (IoT), machine learning (ML).

I. INTRODUCTION

THE LAST decade has seen exciting advancements in fields, such as machine learning (ML) and Internet of Things (IoT). These advancements have driven significant industrial interest in using these technologies to solve several real-world problems [1]. Specifically, the benefits derived from IoT devices have prompted a new industrial age, which is often termed Industry 4.0 [2]. Smart devices are an integral component of Industry 4.0, as they help provide massive

amounts of data which can reveal useful insights. This is usually achieved by processing the data generated by smart devices with artificial intelligence (AI) and ML techniques such as deep learning.

The data generated by IoT devices have a wide range of applications and corresponding analysis techniques. A common analytic approach used in IoT applications is that of classification. Examples of ML-based classification tasks for IoT devices include smart home applications, such as activity detection [3] and energy management [4], security applications, such as detection of unauthorized devices [5], smart city applications, such as noise classification [6] and vehicular traffic monitoring [7], and smart grid applications, such as load classification [8] and detecting anomalous meter readings or energy theft [9]. The use of ML-based classification techniques in these applications has demonstrated improvements in system performance and efficiency. Consequently, there is increasing interest in the development of ML-based solutions for a wide range of IoT applications.

Motivation: While ML-based data analysis techniques bring various advantages, they are also susceptible to attacks that can have severe consequences on the application using them. Recent research has shown that neural networks, and deep networks in particular, are susceptible to adversarial manipulation when using image [10] and audio [11] data. While this research has existed for several years, most researchers have only focused on adversarial examples in image, text, and audio domains. The constraints governing adversarial examples are quite different in image or audio recognition problems as compared to data generated and used in IoT applications and, thus, existing approaches are not directly applicable. This motivates the first question addressed in this article: are the deep learning methods used in conjunction with IoT applications such as smart meters also vulnerable to similar adversarial attacks? These adversarial attacks may force datapoints to be misclassified into a specific class, or more generally, reduce the accuracy of an ML classification model. For example, if the datapoint has been adversarially manipulated, a classification model using smart meter data as input may not identify which devices are operational in a smart home. Since this combination of using deep learning to process the data generated by smart devices used in IoT applications will only see a rise in the future, it is important to explore the possibility of adversarial example attacks and develop strategies to defend against them.

Manuscript received January 2, 2021; revised August 30, 2021; accepted December 15, 2021. Date of publication December 27, 2021; date of current version February 4, 2022. This work was supported by the National Research Foundation, Singapore, under its Strategic Capability Research Centers Funding Initiative. (Corresponding author: Biplab Sikdar.)

The authors are with the Department of Electrical and Computer Engineering, National University of Singapore, Singapore (e-mail: abhijit.singh@u.nus.edu; bsikdar@nus.edu.sg).

Digital Object Identifier 10.1109/JIOT.2021.3138541

This article addresses the problem of adversarial attack generation and mitigation in IoT environments while focusing on smart home applications. Smart homes represent one of the most promising applications of IoT and are expected to become ubiquitous in the forthcoming future. In addition to the improved quality of life for the residents, there are strong business reasons for this as well. For example, an impact assessment study commissioned by the U.K. government found that the adoption of smart meters would lead to annual savings of £4.6 billion in the U.K., due to reduced and more efficient energy consumption [12]. Among other smart devices, these homes will have smart meters, which inform users which appliances are consuming the most amount of electricity. Behavioral research suggests that consumers are better able to manage their electricity consumption when given appliance-by-appliance information [13]. Data from smart meters might even be used by suppliers to monitor the usage patterns of their consumers and adjust their electrical production accordingly, or institute an appliance-specific dynamic pricing regime. Such applications will require sophisticated AI methods such as deep-learning-based nonintrusive load monitoring (NILM) from aggregated smart meter data.

This article presents a methodology to demonstrate that adversarial examples can be obtained for data generated by smart meters, and that they are indistinguishable from true datapoints in terms of their statistical properties. Our work is the first to explore adversarial attacks on deep learning techniques used in IoT-based smart home environments. We develop a white-box attack to generate adversarial datapoints for our use case scenarios (based on the publicly available United Kingdom-domestic appliance-level electricity (UK-DALE) data set [14] and REFIT (Personalized Retrofit Decision Support Tools For U.K. Homes Using Smart Home Technology) data set [15]). We also evaluate the impact of defence strategies for white-box attacks on the proposed adversarial attack mechanism and demonstrate that while the defence strategies reduce the potency of the adversarial attack, the original deep learning models are still significantly affected by it. To summarize, the contributions of this article are as follows.

- 1) We develop a methodology to generate adversarial examples for smart meter data for use on smart home applications.
- 2) We present an analysis to show that outlier detection methods will not be able to combat these adversarial examples. This demonstrates a need for training and deploying ML models for IoT applications which are robust against such adversarial attacks.
- 3) We demonstrate that while a white-box defence method helps the target models, they still remain significantly affected by our adversarial attack.

The remainder of this article is organized as follows. The next section provides a brief review of related work. Section III describes the classification task, data sets used, system and model details, and our proposed adversarial attack mechanism. Section IV contains experimental results of our adversarial attack. In Section V, we present results of the efficacy of the adversarial attack when a white-box defence method is used

to protect the targeted models. Section VI analyzes the results and discusses some limitations and possible future extensions. Section VII concludes this article.

II. RELATED WORK

In the field of computer vision, there is extensive literature on different types of adversarial attacks when using images from popular data sets, such as MNIST [16], ImageNet [17], and CIFAR-10 [18] as inputs. Szegedy *et al.* [10] were the first to show that deep neural networks for image classification were sensitive to small, targeted perturbations. They formulated their optimization problem as a search for an adversarial image, which had minimal distortion compared to the original image, and they used the L-BFGS algorithm [19] to solve this optimization problem. As this is a quasi-Newton method which requires computing a Hessian matrix, this attack is very resource intensive.

Goodfellow *et al.* [20] proposed a fast gradient sign method (FGSM) which can be seen as a one-step method to generate adversarial examples in a faster and less resource-intensive manner than in [10]. It is a gradient-based method which controls the perturbations by using an L_∞ norm bound of $\|x - x'\|_\infty \leq \epsilon$. Nontargeted adversarial examples (x_{adv}) are generated by maximizing the loss function $F(x, y)$

$$x_{adv} = x + \epsilon \text{sign}(\nabla_x F(x, y)). \quad (1)$$

This strategy was expanded further in [21] to include targeted attacks by modifying (1) and maximizing the loss function of the target class y'

$$x_{adv} = x - \epsilon \text{sign}(\nabla_x F(x, y')). \quad (2)$$

While this FGSM attack is fast on account of requiring just one backpropagation step to generate adversarial examples, it is not very efficient. To improve on this, Kurakin *et al.* [22] proposed an iterative version of FGSM, which they named basic iterative method (BIM).

In the DeepFool method for generating adversarial examples [23], Moosavi-Dezfooli *et al.* specifically focused on the decision boundaries of deep neural network image classifiers around a data point x . Their method of generating adversarial examples assumed that each class in a neural network classifier can be separated from other classes by a hyperplane. Their experiments showed that most of the test datapoints were close to the decision boundaries, thus making them vulnerable to small perturbations.

In the AI/ML in IoT domain, there are only a few papers which have considered adversarial examples. Ibitoye *et al.* [24] studied FGSM [20], BIM [22] and projected gradient descent (PGD) [25] adversarial attacks on feed-forward neural networks (FNNs) and self-normalizing neural networks (SNNs) used for classifying intrusion attacks on IoT networks. They used the BoT-IoT data set [26] and found that while FNN outperformed SNN on the intrusion detection in IoT networks task, the SNN was more robust than FNN when adversarial examples were tested.

Rahman *et al.* [27] considered image data sets in IoT applications, and focused on threats from adversarial examples to

TABLE I
COMPARISON OF THE MAIN PAPERS DISCUSSED IN THIS SECTION

Paper	Algorithm	Dataset	Domain
[10]	L-BFGS	MNIST, ImageNet	Image
[20]	One-shot FGSM	MNIST, ImageNet	Image
[22]	Iterative FGSM	ImageNet	Image
[23]	DeepFool	MNIST, ImageNet CIFAR-10	Image
[24]	FGSM, BIM PGD	BoT-IoT	Network Traffic
[27]	L-BFGS, FGSM DeepFool, BIM PGD	CORD	Image

Covid-19 deep learning systems used in medical IoT devices. They tested various Covid-19 diagnostic methods that use deep learning with relevant adversarial examples generated from a variety of attacks, such as L-BFGS [10], FGSM [20], DeepFool [23], BIM [22], and PGD [25] on the CORD data set [28], and found that these models were not robust to adversarial perturbations. Similarly, Ding *et al.* [29] also used image-based inputs, but their work used adversarial perturbations to enhance privacy-preservation of photographs stored on IoT devices. They neglected the unimportant pixels of images to reduce image distortions of adversarial examples, and their experimental results show that their method can fool neural network classifiers with only small changes in visual effects.

Based on the existing literature, we note that, to the best of our knowledge, there is no existing work that has studied the possibility of generating adversarial examples in IoT applications in smart homes. Due to the difference in the data generated by these devices as compared to images considered in the existing literature, the attacker's strategy during the attack can have very different constraints. Thus, our work is novel and offers a useful contribution in an area of increasing importance, as it is crucial to understand the different aspects of security of AI and ML models used to analyze different types of IoT data sets.

Table I provides a comparative summary of the main papers discussed in this section in terms of the algorithms and data sets used by them, along with the domain of their data set.

III. ATTACK METHODOLOGY

This section presents the proposed methodology for generating adversarial examples in smart home scenarios using smart meter data. We start with an overview of the ML task, the data sets used, the system and adversary model, followed by the proposed methodology.

A. Appliance Classification Task

The increasing deployment of smart meters in homes has enabled the collection of fine-grained power consumption data that can be used for a number of applications. Smart meter data can be used to detect and classify the appliances being used in a household that can then facilitate: 1) dynamic pricing mechanisms and methods for demand side management; 2) improved load forecasting; 3) consumer profiling and classification; and 4) feedback to consumers on their usage patterns. In addition, appliance usage data may be used for inferring the behavioral and occupancy patterns of the members of the household, and

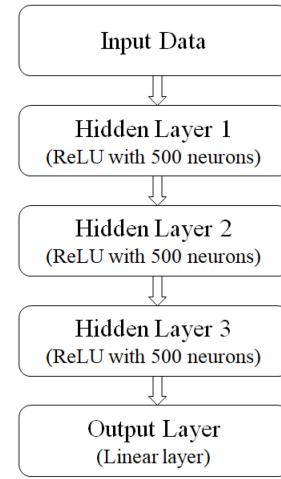


Fig. 1. Model architecture used for the appliance classification task in [32].

used for purposes, such as commercial, emergency, marketing, and law enforcement services [30]. Traditionally, NILM and nonintrusive appliance load monitoring (NIALM) techniques for disaggregating the power consumption into that of individual appliances are based on statistical techniques, such as maximum likelihood estimation and change detection [31].

More recently, ML techniques have been used for appliance classification using smart meter data. This article considers the deep-learning-based classification techniques proposed in [32] and [33], whose effectiveness has been demonstrated on the UK-DALE [14] and REFIT [15] data sets, respectively. The strategy for NILM proposed in [32] is aimed at household appliance identification using residential smart meter data and is based on building a deep learning model to perform this classification task, with substantial preprocessing of data. In [32], the appliance classification task is transformed into a binary classification task, which is the standard practice in this area because existing methods cannot reliably extract appliance profiles of multiple appliances from smart meter data at the same time. Thus, the data first need to be pre-processed to obtain the inputs that will be used in the deep learning model. This involves extracting appliance profiles of the required appliances from the aggregate data one at a time (by treating the rest of the input as noise), followed by training a model to predict whether that particular appliance was in use or not. So essentially, a new model has to be trained to classify each appliance. The NILM strategy proposed in [33] is very similar, and the only differences are found in some aspects of preprocessing the data set. Readers interested in the details of the preprocessing applied can refer to [32] and [33].

Fig. 1 shows the basic architecture of the classification models used in [32]. This architecture remained the same for all the appliances considered from the UK-DALE data set. The deep learning model used three hidden layers with 500 neurons in each layer, followed by a linear output layer. The ReLU activation function [34] was used in the hidden layers. The loss function was a log-softmax function, and the optimizer used was Adam [35] with a learning rate of 0.001. Since the model used for the appliance classification task is not the primary focus of this article, interested readers can refer to

TABLE II
NUMBER OF TEST DATAPPOINTS (TEST DP) FOR EACH APPLIANCE
(AFTER PREPROCESSING STEPS OF [32] AND [33])

[32] (uses UK-DALE dataset)		[33] (uses REFIT dataset)	
Appliance	Test DP	Appliance	Test DP
Kettle	1000	Washing machine	1000
Vacuum cleaner	1000	Dishwasher	1000
Microwave	1000	Microwave	1000

our code at [36] for any further details (or to reproduce the results presented in this article).

The classification models used for all appliances considered in [33] were WaveNet models [37] trained on the REFIT data set. They had 32 residual and dilation channels, and 32 skipped channels as well. Binary cross-entropy-loss function was used with the Adam optimizer with a learning rate of 0.001.

B. Data Set

This article uses two data sets to evaluate the proposed adversarial attack generation and defence mechanisms. The first data set is the UK-DALE data set [14], which was used by Devlin and Hayes [32] to test their deep-learning-based NILM method. This publicly available data set has aggregated and disaggregated appliance data from five households in London, collected over several years. For each house, the whole-house mains power demand and appliance-level power demand is recorded every 6 s using a smart meter.

We note that a large fraction of the data comes from just a single house. House 1 has data from November 2012 to April 2017, which is a period of roughly four and a half years. Houses 2–5 have significantly lesser data, which is discontinuous in patches (up to two months long). House 2 has data from March 2013 to October 2013, House 3 has just one month of data (March 2013), House 4 also has data from March 2013 to October 2013, while House 5 has data from July 2014 to November 2014.

Three appliances from this data set are presented for analysis in this article: 1) kettle; 2) vacuum cleaner; and 3) microwave. All three of these appliances have labeled data available in the UK-DALE data set.

The second data set used in this article is the REFIT data set [15], which was used by Jiang *et al.* [33] to test their deep-learning-based NILM method. This publicly available data set has aggregated and disaggregated appliance data from 20 households in London, collected over two years. For each house, the whole-house mains power demand and appliance-level power demand is timestamped and sampled at 8-s intervals.

Three appliances from this data set are presented for analysis in this article: 1) washing machine; 2) dishwasher; and 3) microwave. All three of these appliances have labeled data available in the REFIT data set.

Table II presents the number of test datapoints for each appliance after the preprocessing steps of [32] and [33].

C. System Model

In this article, we consider a system where smart meters in residential houses send their data to an AI-enabled processing

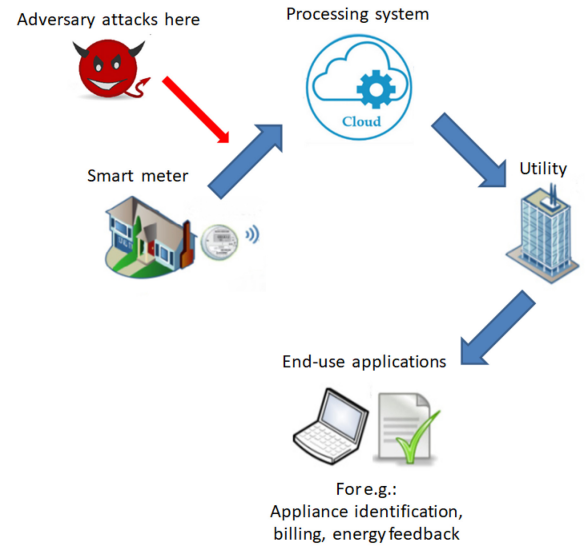


Fig. 2. Diagrammatic version of an adversary in a smart meter system.

system, which may be cloud-based like most commercial processing systems nowadays. This processed data are then sent to the utility provider, which may then use this information for diverse end-use applications as mentioned in Section III-A. Fig. 2 shows a diagrammatic version of the system model and where the adversary fits in.

D. Adversary Model

This article considers a practical adversary or threat model for deep learning algorithms used for classification tasks with smart meter data. The threat model is as follows.

- 1) The adversary can arbitrarily modify the smart meter readings. Such attacks are possible when the adversary has physical access to the meter, and/or it has compromised the encryption keys used (if any) and it has compromised any of the network elements.
- 2) We consider a white-box attack where the adversary has access to the deep learning model. Such attacks are possible, for instance, in the case of insider attacks. Additionally, the white-box model provides a better evaluation of the robustness of the defence mechanism and gives an upper bound on the performance of the adversary.

E. Adversarial Attack Mechanism

To generate adversarial examples, the optimization to be performed by the adversary is formally framed as follows. Assume that we have a function $f(x)$ which classifies the input $x \in R^d$ (a d -dimensional real-valued vector) into one of k possible classes. The adversary seeks to add a perturbation r to a datapoint x so that it is misclassified. Then, the objective of the adversary can be modeled as

$$\min \|r\|_2 \quad (3)$$

$$\text{subject to: } f(x + r) = k_{\text{adversarial_target}} \quad (4)$$

where $k_{\text{adversarial_target}}$ is not equal to $k_{\text{predicted_class}}$, because equality would make it trivial. The formulation above is

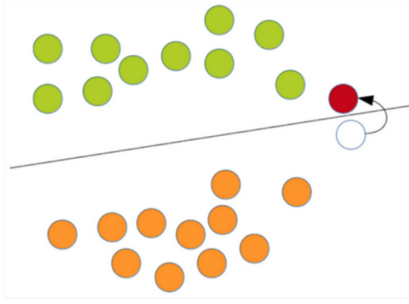


Fig. 3. Intuition behind this adversarial attack on a toy problem. This figure assumes 2-D linearly separable data on a binary classification task. The goal of the attack is to craft an adversarial example that crosses the target model's decision boundary. Figure from [38].

Algorithm 1 Generating an Adversarial Datapoint

Input: True datapoint (X), Target class ($target_y$), Trained model ($model$)

Output: Adversarial datapoint ($X_{fooling}$)

Other variables: Learning Rate ($learning_rate$), Max no. of iterations (max_iter), Class scores generated by $model$ ($model_score$), Predicted label ($predicted_label$), Score of target class ($target_score$), Gradient of $X_{fooling}$ (dx)

```

1: function GEN_ADV_DP( $X$ ,  $target\_y$ ,  $model$ )
2:   Use model in evaluation/inference mode
3:   Initialize adversarial datapoint (copy of true data-
     point:  $X_{fooling} = X$ )
4:   Set  $learning\_rate$  and  $max\_iter$ 
5:   for  $i < max\_iter$  do  $\triangleright$  (loop for gradient ascent)
6:     Compute  $model\_score$  of  $X_{fooling}$ 
7:     Extract  $predicted\_label$  from  $model\_score$ 
8:     if  $predicted\_label = target\_y$  then
9:       break from for-loop
10:    else
11:      Extract  $target\_score$  from  $model\_score$ 
12:      Perform backpropagation on  $target\_score$ 
13:      Extract gradient update of  $X_{fooling}$  ( $dx$ )
14:      Normalize the gradient update:
          $dx = learning\_rate \times (dx / \text{norm}(dx))$ 
15:      Update  $X_{fooling}$  with normalized  $dx$ :
          $X_{fooling} += dx$   $\triangleright$  (gradient ascent)
16:      Clear current gradients
17:    end if
18:  end for
19: end function
20: Return  $X_{fooling}$ 

```

aimed to find the smallest adversarial perturbation r which would classify the datapoint x into a target class of the adversary's choice. Since the models targeted in this article use a binary classification scheme, the optimization in the problem above seeks to flip the true class of the datapoint. Fig. 3 gives an intuition behind this adversarial attack on a low-dimensional toy problem (this figure assumes 2-D data on a binary classification task that is linearly separable).

The overall algorithm for executing the adversarial attack mechanism is shown in Algorithm 1. It can be thought of as gradient ascent on the target class, because we are performing a gradient ascent using the model output to execute the above

TABLE III
COMPARISON OF $F1$ SCORES ACHIEVED BY THE CLASSIFIERS
IN OUR IMPLEMENTATION AND THOSE IN [32]

Appliance	[32]	Current
Kettle	0.76	0.81
Vacuum cleaner	0.69	0.70
Microwave	0.84	0.82

optimization. One of the key steps in this algorithm is step 6. The model score contains real-valued numbers associated with each class. The predicted class is the class which has the highest score associated with it in the $model_score$ array. If the predicted class is not the same as the desired target class (step 8), then we extract the real-valued number associated with the desired target class from the $model_score$ array, and perform backpropagation (step 12). Then, we use the normalized gradient updates to nudge the adversarial datapoint in the desired direction iteratively (steps 13–16), until (4) is satisfied (or we exceed max_iter).

IV. EVALUATION OF ATTACK STRATEGY

This section presents an experimental evaluation of the adversarial attack strategy proposed in Algorithm 1. The results for the two data sets are presented in distinct sections. Within the section for each data set, we first validate our implementation of the deep-learning-based appliance classification methods by comparing our results with those reported in [32] and [33].

Next, we report the number of test datapoints for which we were able to generate adversarial examples. This is followed by the histogram of magnitudes and angle with respect to a chosen unit vector for each of the datapoints, when they are interpreted as a vector in a high-dimensional space. The unit vector we chose had the same magnitude for each dimension. In general, the magnitude of each dimension is given by $(1/\sqrt{d})$, where d is the number of dimensions. So for 2-D data, the unit vector would be $([1/\sqrt{2}]\hat{i} + [1/\sqrt{2}]\hat{j})$. This scheme of selecting a unit vector was chosen because our objective is to understand and visualize the distribution of the datapoints with respect to a fixed point, and such a unit vector has nonzero magnitudes in all dimensions, serving our needs perfectly. In each figure, the green histograms represent true datapoints, and the red histograms represent adversarial datapoints. Subfigures (a) and (c) are the histograms of magnitudes for true and adversarial datapoints, respectively, and subfigures (b) and (d) are the histograms of angle with respect to the chosen unit vector, for true and adversarial datapoints, respectively. All subfigures within each figure have the same range on x -axis and y -axis to facilitate easier comparisons.

A. UK-DALE Data Set

Table III compares the $F1$ scores achieved by the deep-learning-based appliance classification method proposed by Devlin and Hayes [32] on the test data set, and our reimplementation of their work. For all appliances, there is a close match between the classification results achieved in our implementation and those reported in [32]. The small differences in the classification performance metrics are due to the use

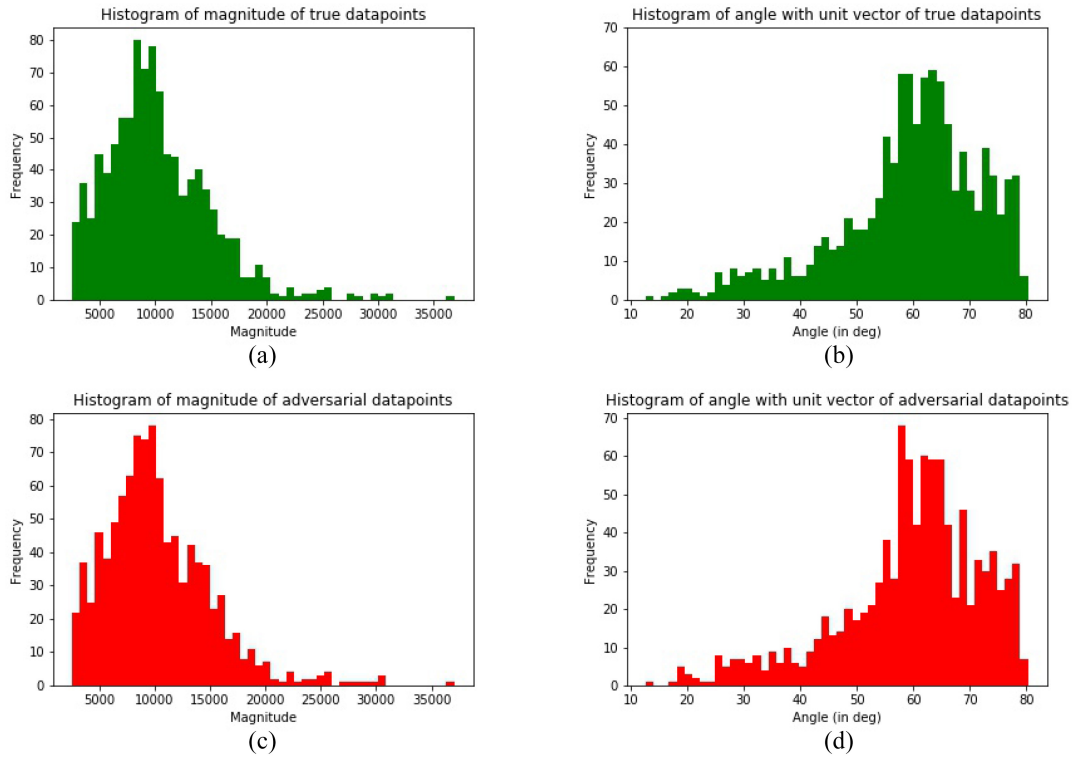


Fig. 4. Histogram of (a) and (c) magnitudes and (b) and (d) angle with unit vector of true datapoints (green histograms) and adversarial datapoints (red histograms) (when interpreted as a vector in a high-dimensional space). Appliance: Kettle and data set: UK-DALE.

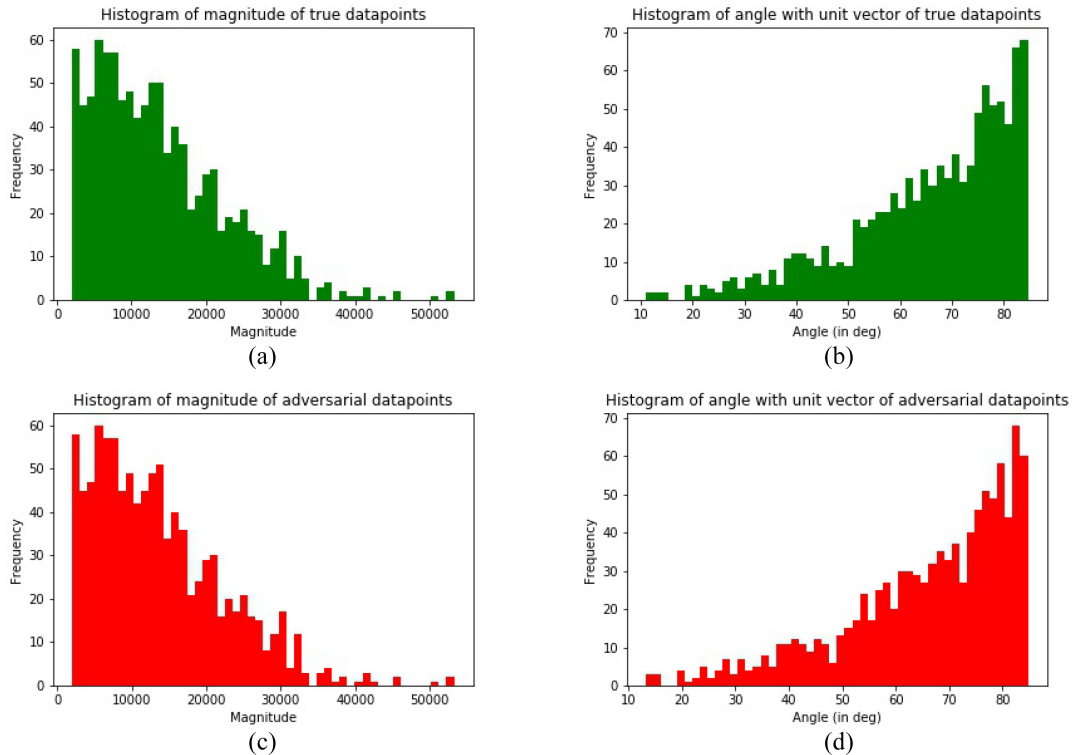


Fig. 5. Histogram of (a) and (c) magnitudes and (b) and (d) angle with unit vector of true datapoints (green histograms) and adversarial datapoints (red histograms) (when interpreted as a vector in a high-dimensional space). Appliance: Vacuum cleaner and data set: UK-DALE.

of different random seeds used for initialization of weights, as [32] did not fix a random seed.

For all appliances, the proposed methodology is able to generate adversarial examples for *all 1000 test datapoints*.

Figs. 4–6 capture the trend of all the true and adversarial datapoints for each of the appliances, respectively. These figures show the histogram of magnitudes and angle with respect to a chosen unit vector for each of the datapoints, when they

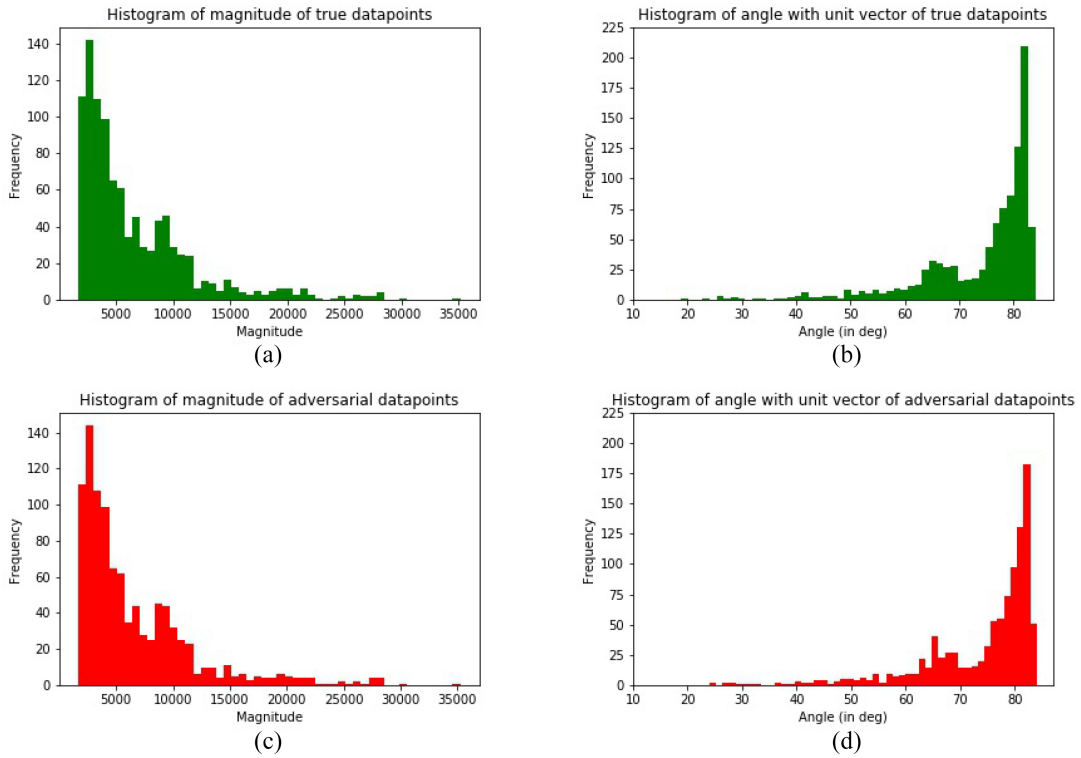


Fig. 6. Histogram of (a) and (c) magnitudes and (b) and (d) angle with unit vector of true datapoints (green histograms) and adversarial datapoints (red histograms) (when interpreted as a vector in a high-dimensional space). Appliance: Microwave and data set: UK-DALE.

TABLE IV
COMPARISON OF $F1$ SCORES ACHIEVED BY THE CLASSIFIERS IN OUR IMPLEMENTATION AND THOSE IN [33]

Appliance	[33]	Current
Washing machine	0.76	0.72
Dishwasher	0.56	0.60
Microwave	0.49	0.53

are interpreted as a vector in a high-dimensional space. We note that there is a close match between the histograms of the true and adversarial datapoints.

B. REFIT Data Set

Table IV compares the $F1$ scores achieved by the deep-learning-based appliance classification method proposed by Jiang *et al.* [33] on the test data set, and our reimplementation of their work. For all appliances, our implementation has a close match to the classification results reported in [33]. The minor differences are due to the different random seeds used (as [33] did not use any random seed), and because we had to plug in some hyperparameter values as all were not specified.

For all appliances, the proposed methodology is able to generate adversarial examples for *all 1000 test datapoints*. Figs. 7–9 capture the trend of all the true and adversarial datapoints for each of the appliances, respectively. These figures show the histogram of magnitudes and angle with respect to a chosen unit vector for each of the datapoints, when they are interpreted as a vector in a high-dimensional space. Yet again we note that there is a close match between the histograms of the true and adversarial datapoints.

TABLE V
ASPECTS OF MODEL INFORMATION AVAILABLE TO THE ADVERSARY IN DIFFERENT SETTINGS (“HIDDEN NEURONS”: NUMBER OF HIDDEN NEURONS IN EACH LAYER, “LAYERS”: NUMBER OF LAYERS IN THE MODEL, AND “ACTIVATION FN”: ACTIVATION FUNCTION USED IN EACH LAYER). DATA SET: UK-DALE

Setting	Hidden neurons	Layers	Activation fn
White-box	✓	✓	✓
Partial 1	×	✓	✓
Partial 2	×	×	✓
Black-box	×	×	×

V. DEFENCE AGAINST WHITE-BOX ATTACK

White-box attacks rely on the adversary having access to the complete model information. One of the defences to protect ML models from white-box adversarial attacks is to restrict the amount of model information available to the adversaries. In this section, we present our results on the efficacy of the adversarial attack when varying levels of information is available to the adversary about the target model.

A. UK-DALE Data Set

Table V summarizes the different scenarios considered while evaluating the defence against white-box attacks on the models trained on the UK-DALE data set. In the white-box setting, all model information are available. In the Partial 1 setting, the adversary knows what the general architecture of the model is (i.e., number of layers and activation function), but does not know how many hidden neurons are present in each layer. In the Partial 2 setting, the adversary only knows which activation function has been used, with no knowledge about the rest of the model architecture (i.e., number of layers and number of

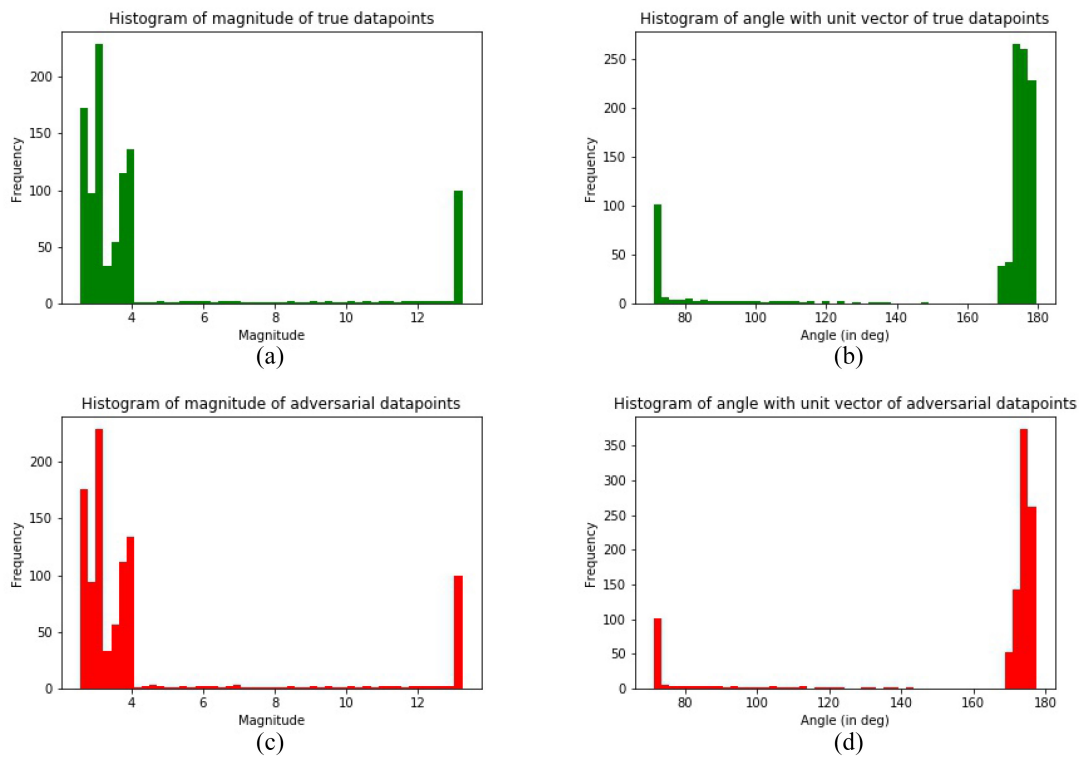


Fig. 7. Histogram of (a) and (c) magnitudes and (b) and (d) angle with unit vector of true datapoints (green histograms) and adversarial datapoints (red histograms) (when interpreted as a vector in a high-dimensional space). Appliance: Washing machine and data set: REFIT.

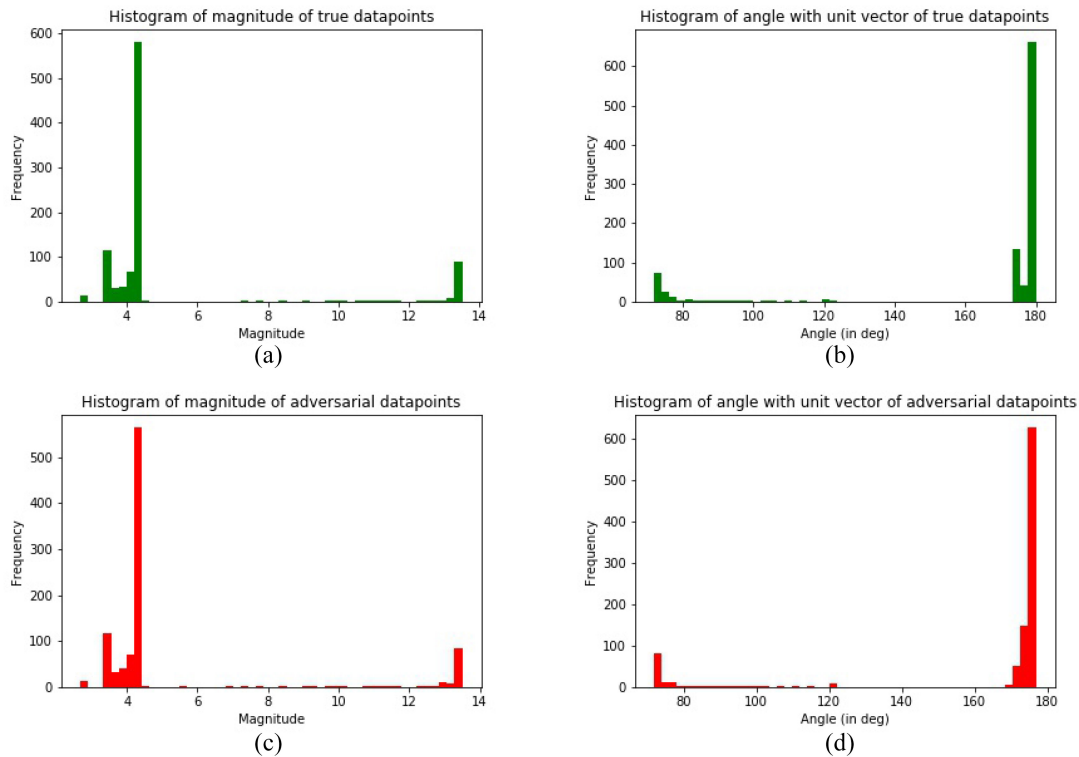


Fig. 8. Histogram of (a) and (c) magnitudes and (b) and (d) angle with unit vector of true datapoints (green histograms) and adversarial datapoints (red histograms) (when interpreted as a vector in a high-dimensional space). Appliance: Dishwasher and data set: REFIT.

hidden neurons in each layer). In the black-box setting, the adversary has no information about the target model.

To evaluate the effectiveness of the various settings against the proposed methodology for generating adversarial attacks

on deep-learning-based appliance classification, we conducted a simulation-based study whose experiments were structured as follows. To simulate the different levels of information that the adversary has, we trained a new model for each

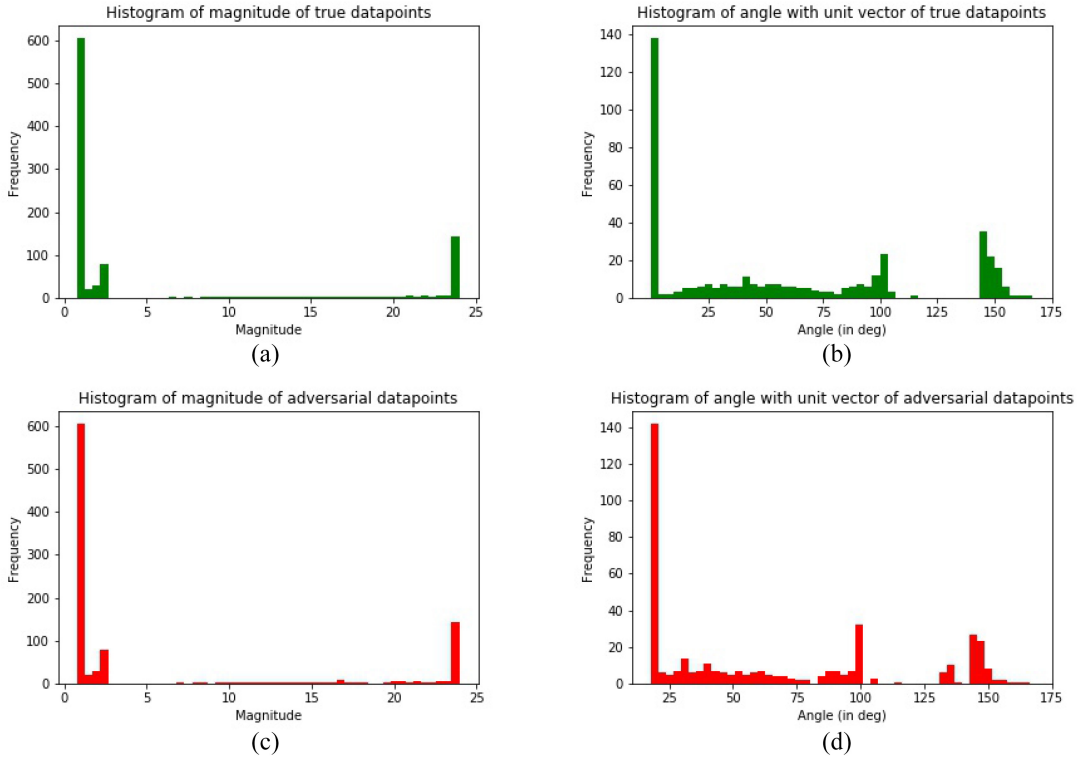


Fig. 9. Histogram of (a) and (c) magnitudes and (b) and (d) angle with unit vector of true datapoints (green histograms) and adversarial datapoints (red histograms) (when interpreted as a vector in a high-dimensional space). Appliance: Microwave and data set: REFIT.

setting. Each of these new models was trained using the same training set as the original model. Then, using the same adversarial attack presented in Algorithm 1, we generated adversarial examples for each of the test datapoints, using the newly-trained model as a proxy for the actual target model. Finally, we used the adversarial datapoints generated in this manner as test datapoints on the actual target model. In our results, we report the number of adversarial datapoints generated using the proxy models, which were able to successfully fool the original model (which is the target model).

Table VI compares the results obtained for each of the appliances. Each appliance had 1000 test datapoints, as mentioned in Table II. We ran the experiments several times (details in the Appendix) for each setting, and report the mean and standard deviation of the number of adversarial datapoints which could successfully fool the original model. This was done to ensure that the results obtained were not down to finding a particularly good or particularly bad local optimum. The Appendix has further details on the different configurations used in each setting.

To judge the effectiveness of an adversarial attack or defence method, it is often easier to think in terms of the error rate of the model. A model with 80% accuracy has an error rate of 20%. A successful adversarial attack will push up the error rate significantly. Similarly, a good defence method will be able to bring down the error rate from the levels achieved by an adversarial attack. To illustrate this, Fig. 10 presents the error rates under attack and no-attack scenarios. In Fig. 10, the blue bars correspond to the error rate when the original model is used for appliance classification. The orange bars correspond to the error rate when the original model is attacked using the

TABLE VI
NUMBER OF ADVERSARIAL DATAPPOINTS GENERATED USING PROXY MODELS WHICH COULD FOOL THE ORIGINAL MODEL. MEAN AND STANDARD DEVIATION OF SEVERAL RUNS IS REPORTED (DETAILS IN THE APPENDIX). GUIDE TO ACRONYMS: APPLIANCES (APPL.), KETTLE (KT), VACUUM CLEANER (VC), MICROWAVE (MW), WHITE-BOX SETTING (WB), PARTIAL 1 SETTING (P1), PARTIAL 2 SETTING (P2), AND BLACK-BOX SETTING (BB).
DATA SET: UK-DALE

Appl.	WB	P1	P2	BB
KT	1000	553 $\pm$ 96	566 $\pm$ 87	593 $\pm$ 84
VC	1000	434 $\pm$ 47	425 $\pm$ 48	403 $\pm$ 38
MW	1000	539 $\pm$ 112	530 $\pm$ 87	527 $\pm$ 73

proposed attack mechanism (results presented in Section IV). Finally, the green bars correspond to the error rate when the model is adversarially attacked after we have implemented the white-box defence mechanism explained in this section. We note that the defence mechanism is able to significantly decrease the effectiveness of the adversarial attack. However, the defence mechanism is unable to bring the error rates down to those in the no-attack scenario.

B. REFIT Data Set

The flow and structure of the experiments done on the REFIT data set were almost exactly the same as those explained in the previous section. Thus, in this section, we only explain the few things that differed.

Table VII summarizes the different scenarios considered while evaluating the defence against white-box attacks on the models trained on the REFIT data set. In the white-box setting, all model information is available. In the Partial 1 setting, the adversary knows most of the information about the WaveNet

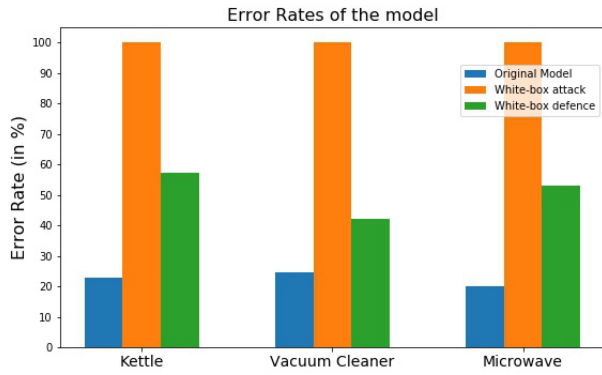


Fig. 10. Error rates of the model in its original state (blue), after it is adversarially attacked without any defence mechanism (orange), and after it is adversarially attacked after implementing white-box defence (green). Data set: UK-DALE.

TABLE VII

ASPECTS OF MODEL INFORMATION AVAILABLE TO THE ADVERSARY IN DIFFERENT SETTINGS (“R/D CHANNELS”: NUMBER OF RESIDUAL AND DILATION CHANNELS, “SKIP CHANNELS”: NUMBER OF SKIPPED CHANNELS, AND “MODEL ARCH.”: MODEL ARCHITECTURE BEING USED). DATA SET: REFIT

Setting	R/D Channels	Skip Channels	Model Arch.
White-box	✓	✓	✓
Partial 1	×	✓	✓
Partial 2	×	×	✓
Black-box	×	×	×

TABLE VIII

NUMBER OF ADVERSARIAL DATAPPOINTS GENERATED USING PROXY MODELS WHICH COULD FOOL THE ORIGINAL MODEL. MEAN AND STANDARD DEVIATION OF SEVERAL RUNS IS REPORTED (DETAILS IN THE APPENDIX). GUIDE TO ACRONYMS: APPLIANCES (APPL.), WASHING MACHINE (WM), DISHWASHER (DW), MICROWAVE (MW), WHITE-BOX SETTING (WB), PARTIAL 1 SETTING (P1), PARTIAL 2 SETTING (P2), AND BLACK-BOX SETTING (BB). DATA SET: REFIT

Appl.	WB	P1	P2	BB
WM	1000	836 ± 81	773 ± 61	763 ± 107
DW	1000	929 ± 44	946 ± 8	906 ± 18
MW	1000	723 ± 4	726 ± 10	732 ± 16

model being used, but does not know the number of residual and dilation channels that are being used. In the Partial 2 setting, the adversary does not know the number of skipped channels as well. In the black-box setting, the adversary has no information about the target model. We use a bidirectional gated recurrent unit (BGRU) model [39] as the proxy model for this setting.

In our results, we report the number of adversarial datapoints generated using the proxy models, which were able to successfully fool the original model (which is the target model). Table VIII compares the results obtained for each of the appliances. Each appliance had 1000 test datapoints, as mentioned in Table II. We ran the experiments several times (details in the Appendix) for each setting, and report the mean and standard deviation of the number of adversarial datapoints which could successfully fool the original model. The Appendix has further details on the different configurations used in each setting.

Fig. 11 presents the error rates under attack and no-attack scenarios. In Fig. 11, the blue bars correspond to the error

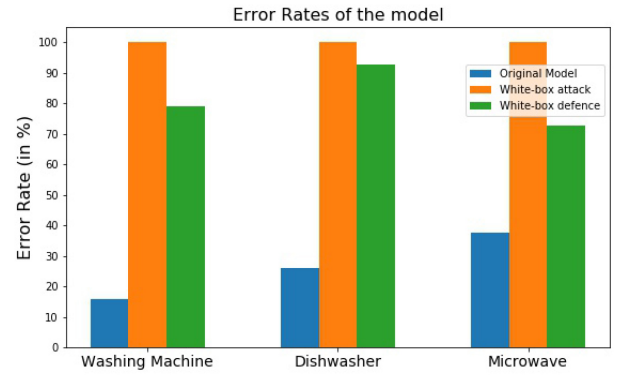


Fig. 11. Error rates of the model in its original state (blue), after it is adversarially attacked without any defence mechanism (orange), and after it is adversarially attacked after implementing white-box defence (green). Data set: REFIT.

rate when the original model is used for appliance classification. The orange bars correspond to the error rate when the original model is attacked using the proposed attack mechanism (results presented in Section IV). Finally, the green bars correspond to the error rate when the model is adversarially attacked after we have implemented the white-box defence mechanism explained in this section. We note again that the defence mechanism is able to decrease the effectiveness of the adversarial attack. However, the defence mechanism is unable to bring the error rates down to those in the no-attack scenario.

VI. DISCUSSION

The results in Tables III and IV confirmed that the classifiers implemented in our code had similar performances as compared to the models proposed by Devlin and Hayes [32] and Jiang *et al.* [33]. Since it is not possible to visualize high-dimensional spaces, we interpreted each datapoint as a vector in that high-dimensional space. Then, we compared the magnitude profile and angle made with a suitable unit vector profile for both the true and adversarial datapoints. Our results in Figs. 4–9 showed that the true datapoints and adversarial datapoints had very similar profiles, and almost identical ranges for both magnitude and angle, which confirmed that the true and adversarial datapoints exist in the same high-dimensional space. This finding was important because it showed that these adversarial datapoints were not outliers. Thus, outlier detection and filtering methods will not be able to tackle the problem posed by these adversarial datapoints. This highlights the need for deploying adversarially robust models for processing data generated by IoT devices such as smart meters. Furthermore, if on-device computations are necessary for a particular application due to concerns such as privacy of data, then these adversarially robust models must not be too computationally intensive to be deployed on edge devices.

The results in Section V, and particularly Figs. 10 and 11, demonstrated that white-box defence methods help to reduce the severity of the adversarial attack. After implementing the defence, the error rate of the target models went down from 100% (orange bars) for all appliances to about 40%–60% (green bars) in Fig. 10, and about 70%–90% (green bars) in Fig. 11. While this shows the effectiveness of the defence method, when we compare with the error rates of the original

models (blue bars), we find that the adversarial attack was still able to increase the error rate by a sizeable margin (blue bars versus green bars). This result tilts the scale in favor of the proposed adversarial attack.

There are a few limitations of this study which warrant a discussion as well, and they tie in with possible future extensions. One future avenue to explore would be to try black-box adversarial attack mechanisms which only require access to the inference-time predictions of a model, and investigate their efficacy. In this article, we use a white-box attack, which assumes access to the trained model to be able to execute the attack. This assumption may not always be valid. In any case, this work demonstrates that making the models used for such tasks publicly available can result in undesirable consequences.

VII. CONCLUSION

This article demonstrated that deep learning models used for classification tasks on data generated by smart meters are vulnerable to adversarial examples, and that it is possible to develop adversarial datapoints that are statistically indistinguishable from true datapoints. We developed a gradient ascent strategy for generating adversarial datapoints to target classification models proposed in existing literature. We showed that the proposed attack mechanism is able to generate adversarial datapoints for every test datapoint of all the appliances on both the publicly available smart meter data sets. Furthermore, the demonstration that these adversarial datapoints are indistinguishable from the true datapoints means that non ML methods like outlier detection would fail to distinguish them from true datapoints (without significant collateral damage). This points to a need for ML-based solutions. Even when the target models were defended by certain white-box defence methods, our adversarial attack was successful in increasing the error rate by 100%–200%.

APPENDIX

This section presents the details of the experimental settings used for evaluating the impact of white-box defence mechanisms on the proposed methodology for generating adversarial examples.

A. UK-DALE Data Set

For every appliance, the configuration details remained the same. The various configurations used were as follows.

- 1) *Original Model*: Three hidden layers, with 500 neurons in each layer. ReLU activation function used in each layer.
- 2) *Partial 1 Model*: This model had four different configurations, each of which had three hidden layers with a ReLU activation function. Each configuration differed only in the number of neurons in each layer. Each configuration was run five times (so 20 runs (per appliance) in total for Partial 1 model).
 - a) *Config 1*: 200 neurons in each layer.
 - b) *Config 2*: 800 neurons in each layer.
 - c) *Config 3*: 200, 500, and 800 neurons in layers 1–3, respectively.

- d) *Config 4*: 800, 500, and 200 neurons in layers 1–3, respectively.

- 3) *Partial 2 Model*: This model had seven different configurations, each of which used a ReLU activation function. Each configuration differed either in the number of layers or number of neurons in each layer. Each configuration was run five times (so 35 runs (per appliance) in total for Partial 2 model).

- a) *Config 1*: Two layers, 200 neurons in each layer.
- b) *Config 2*: Two layers, 500 neurons in each layer.
- c) *Config 3*: Two layers, 800 neurons in each layer.
- d) *Config 4*: Four layers, 200 neurons in each layer.
- e) *Config 5*: Four layers, 500 neurons in each layer.
- f) *Config 6*: Four layers, 800 neurons in each layer.
- g) *Config 7*: 200, 400, 600, and 800 neurons in layers 1–4 respectively.

- 4) *Black-Box Model*: This model had six different configurations, each of which differed in aspects, such as the number of layers and activation functions used. Each configuration was run five times (so 30 runs (per appliance) in total for the black-box model).

- a) *Config 1*: Two layers with 300 and 700 neurons in each layer respectively. Tanh activation function.
- b) *Config 2*: Two layers with 300 and 700 neurons in each layer respectively. Leaky_relu activation function.
- c) *Config 3*: Three layers with 300, 500, and 700 neurons in each layer respectively. Tanh activation function.
- d) *Config 4*: Three layers with 300, 500, and 700 neurons in each layer respectively. Leaky_relu activation function.
- e) *Config 5*: Four layers with 200, 400, 600, and 800 neurons in each layer respectively. Tanh activation function.
- f) *Config 6*: Four layers with 200, 400, 600, and 800 neurons in each layer respectively. Leaky_relu activation function.

B. REFIT Data Set

For every appliance, the configuration details remained the same. The various configurations used were as follows.

- 1) *Original Model*: A 6-layer WaveNet model with 32 residual channels, 32 dilation channels, and 32 skipped channels. Kernel size was 3.
- 2) *Partial 1 Model*: A 6-layer WaveNet model with 16 residual channels, 16 dilation channels, and 32 skipped channels. Kernel size was 3. This configuration was run five times for each appliance.
- 3) *Partial 2 Model*: A 6-layer WaveNet model with 16 residual channels, 16 dilation channels, and 16 skipped channels. Kernel size was 3. This configuration was run five times for each appliance.
- 4) *Black-Box Model*: BGRU model with three layers of 64 hidden neurons each. This configuration was run five times for each appliance.

For any further details, interested readers can refer to [33].

ACKNOWLEDGMENT

Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore.

REFERENCES

- [1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [2] H. Lasi, P. Fettke, H.-G. Kemper, T. Feld, and M. Hoffmann, "Industry 4.0," *Bus. Inf. Syst. Eng.*, vol. 6, no. 4, pp. 239–242, 2014.
- [3] B. Das, D. J. Cook, N. C. Krishnan, and M. Schmitter-Edgcombe, "One-class classification-based real-time activity error detection in smart homes," *IEEE J. Sel. Top. Signal Process.*, vol. 10, no. 5, pp. 914–923, Aug. 2016.
- [4] D. Koolen, N. Sadat-Razavi, and W. Ketter, "Machine learning for identifying demand patterns of home energy management systems with dynamic electricity pricing," *Appl. Sci.*, vol. 7, no. 11, p. 1160, 2017.
- [5] V. Thangavelu, D. M. Divakaran, R. Sairam, S. S. Bhunia, and M. Gurusamy, "DEFT: A distributed IoT fingerprinting technique," *IEEE Internet Things J.*, vol. 6, no. 1, pp. 940–952, Feb. 2019.
- [6] Y. Alsouda, S. Pillana, and A. Kurti, "IoT-based urban noise identification using machine learning: Performance of SVM, KNN, bagging, and random forest," in *Proc. Int. Conf. Omni-Layer Intell. Syst.*, 2019, pp. 62–67.
- [7] M. Won, "Intelligent traffic monitoring systems for vehicle classification: A survey," *IEEE Access*, vol. 8, pp. 73340–73358, 2020.
- [8] S.-L. Yang, C. Shen, and S.-L. Yang, "A review of electric load classification in smart grid environment," *Renew. Sustain. Energy Rev.*, vol. 24, pp. 103–110, Aug. 2013.
- [9] R. Punmiya and S. Choe, "Energy theft detection using gradient boosting theft detector with feature engineering-based preprocessing," *IEEE Trans. Smart Grid*, vol. 10, no. 2, pp. 2326–2329, Mar. 2019.
- [10] C. Szegedy *et al.*, "Intriguing properties of neural networks," 2013, *arXiv:1312.6199*.
- [11] N. Carlini and D. Wagner, "Audio adversarial examples: Targeted attacks on speech-to-text," in *Proc. IEEE Security Privacy Workshops (SPW)*, 2018, pp. 1–7.
- [12] F. Lienert and M. Carson, "Smart meter roll-out for the domestic sector (GB)," Dept. Energy Climate Change, Govt. U.K., London, U.K., Rep. DECC0009, Jan. 2014.
- [13] C. Fischer, "Feedback on household electricity consumption: A tool for saving energy?" *Energy Efficiency*, vol. 1, no. 1, pp. 79–104, 2008.
- [14] J. Kelly and W. Knottenbelt, "The UK-DALE dataset, domestic appliance-level electricity demand and whole-house demand from five U.K. homes," *Sci. Data*, vol. 2, no. 1, pp. 1–14, 2015.
- [15] D. Murray, L. Stankovic, and V. Stankovic, "An electrical load measurements dataset of United Kingdom households from a two-year longitudinal study," *Sci. Data*, vol. 4, no. 1, pp. 1–12, 2017.
- [16] L. Deng, "The MNIST database of handwritten digit images for machine learning research [best of the Web]," *IEEE Signal Process. Mag.*, vol. 29, no. 6, pp. 141–142, Nov. 2012.
- [17] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255.
- [18] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," Dept. Comput. Sci., Univ. Toronto, Toronto, ON, Canada, Rep. TR-2009, 2009.
- [19] D. C. Liu and J. Nocedal, "On the limited memory BFGS method for large scale optimization," *Math. Program.*, vol. 45, nos. 1–3, pp. 503–528, 1989.
- [20] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2014, *arXiv:1412.6572*.
- [21] A. Kurakin, I. Goodfellow, and S. Bengio, "Adversarial machine learning at scale," 2016, *arXiv:1611.01236*.
- [22] A. Kurakin, I. Goodfellow, and S. Bengio, "Adversarial examples in the physical world," 2016, *arXiv:1607.02533*.
- [23] S.-M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard, "Deepfool: A simple and accurate method to fool deep neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2574–2582.
- [24] O. Ibitoye, O. Shafiq, and A. Matrawy, "Analyzing adversarial attacks against deep learning for intrusion detection in IoT networks," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, 2019, pp. 1–6.
- [25] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," 2017, *arXiv:1706.06083*.
- [26] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," *Future Gener. Comput. Syst.*, vol. 100, pp. 779–796, Nov. 2019.
- [27] A. Rahman, M. S. Hossain, N. A. Alrajeh, and F. Alsolami, "Adversarial examples—Security threats to COVID-19 deep learning systems in medical IoT devices," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9603–9610, Jun. 2021.
- [28] L. L. Wang *et al.*, "CORD-19: The COVID-19 open research dataset," 2020, *arXiv:2004.10706*.
- [29] X. Ding, S. Zhang, M. Song, X. Ding, and F. Li, "Towards invisible adversarial examples against DNN-based privacy leakage for Internet of Things," *IEEE Internet Things J.*, vol. 8, no. 2, pp. 802–812, Jan. 2021.
- [30] E. McKenna, I. Richardson, and M. Thomson, "Smart meter data: Balancing consumer privacy concerns with legitimate applications," *Energy Policy*, vol. 41, pp. 807–814, Feb. 2012.
- [31] G. W. Hart, "Nonintrusive appliance load monitoring," *Proc. IEEE*, vol. 80, no. 12, pp. 1870–1891, Dec. 1992.
- [32] M. A. Devlin and B. P. Hayes, "Non-intrusive load monitoring and classification of activities of daily living using residential smart meter data," *IEEE Trans. Consum. Electron.*, vol. 65, no. 3, pp. 339–348, Aug. 2019.
- [33] J. Jiang, Q. Kong, M. D. Plumbley, N. Gilbert, M. Hoogendoorn, and D. M. Roijers, "Deep learning-based energy disaggregation and on/off detection of household appliances," *ACM Trans. Knowl. Disc. Data*, vol. 15, no. 3, pp. 1–21, 2021.
- [34] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. ICML*, 2010, pp. 1–8.
- [35] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
- [36] A. Singh and B. Sikdar, "Code Related to the Paper," Dec. 2020. [Online]. Available: <https://www.ece.nus.edu.sg/stfpage/bsikdar/scripts/adversarial>
- [37] A. V. D. Oord *et al.*, "WaveNet: A generative model for raw audio," 2016, *arXiv:1609.03499*.
- [38] "How to Attack Machine Learning." [Online]. Available: <https://towardsdatascience.com/> (Accessed: May 10, 2021).
- [39] A. Graves, A.-R. Mohamed, and G. Hinton, "Speech recognition with deep recurrent neural networks," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2013, pp. 6645–6649.



Abhijit Singh received the B.Tech. degree in electrical and electronics engineering from the Vellore Institute of Technology, Vellore, India, in 2017, and the M.Sc. degree in AI from The University of Edinburgh, Edinburgh, Scotland, in 2018. He is currently pursuing the Ph.D. degree in machine learning with the National University of Singapore, Singapore.

His research focuses on adversarial machine learning.



Biplab Sikdar (Senior Member, IEEE) received the B.Tech. degree in electronics and communication engineering from North Eastern Hill University, Shillong, India, in 1996, the M.Tech. degree in electrical engineering from the Indian Institute of Technology Kanpur, Kanpur, India, in 1998, and the Ph.D. degree in electrical engineering from Rensselaer Polytechnic Institute, Troy, NY, USA, in 2001.

He was a Faculty with Rensselaer Polytechnic Institute, Troy, NY, USA, from 2001 to 2013, first as an Assistant Professor and then as an Associate Professor. He is currently an Associate Professor with the Department of Electrical and Computer Engineering, National University of Singapore, Singapore, where he serves as the Vice Dean of Graduate Program with the Faculty of Engineering and as an Area Director of Communications and Networks Lab. His current research interests include wireless network and security for Internet of Things and cyber-physical systems.

Dr. Sikdar served as an Associate Editor for the IEEE TRANSACTIONS ON COMMUNICATIONS from 2007 to 2012 and IEEE TRANSACTIONS ON MOBILE COMPUTING from 2014 to 2017. He currently serves as an Associate Editor for the IEEE INTERNET OF THINGS JOURNAL.